

- YOLOv7 Detailed Architecture Guide
  - YOLO Evolution (Quick Context)
  - Full YOLOv7 Architecture
  - 1. Input Processing
  - 2. Backbone — ELAN
    - Basic Building Block: Conv-BN-SiLU
    - ELAN Block
    - Backbone Stages
  - 3. Neck — FPN + PAN
    - FPN — Top-Down Path (semantic information flows down)
    - PAN — Bottom-Up Path (spatial information flows up)
    - Final Neck Outputs
  - 4. Head — RepConv + IDetect
    - RepConv (Re-parameterized Convolution)
    - IDetect — The Detection Head
  - 5. Anchor Boxes
  - 6. Prediction Decoding
  - 7. Total Predictions Per Image
  - 8. Loss Function
    - $L_{box}$  — CloU Loss (Complete IoU)
    - $L_{obj}$  — Objectness Loss (BCE with IoU target)
    - $L_{cls}$  — Classification Loss (BCE)
  - 9. Non-Maximum Suppression (NMS)
  - 10. What Your Fine-Tuning Changes
  - 11. Full Number Summary
  - 12. One-Line Summary of Every Component

# YOLOv7 Detailed Architecture Guide

---

**YOLOv7 Gym Equipment Detection | UNH Deep Learning Spring 2026**

**Team:** Varun Gazala · Mohit Raiyani · Jatinkumar Nabhoya

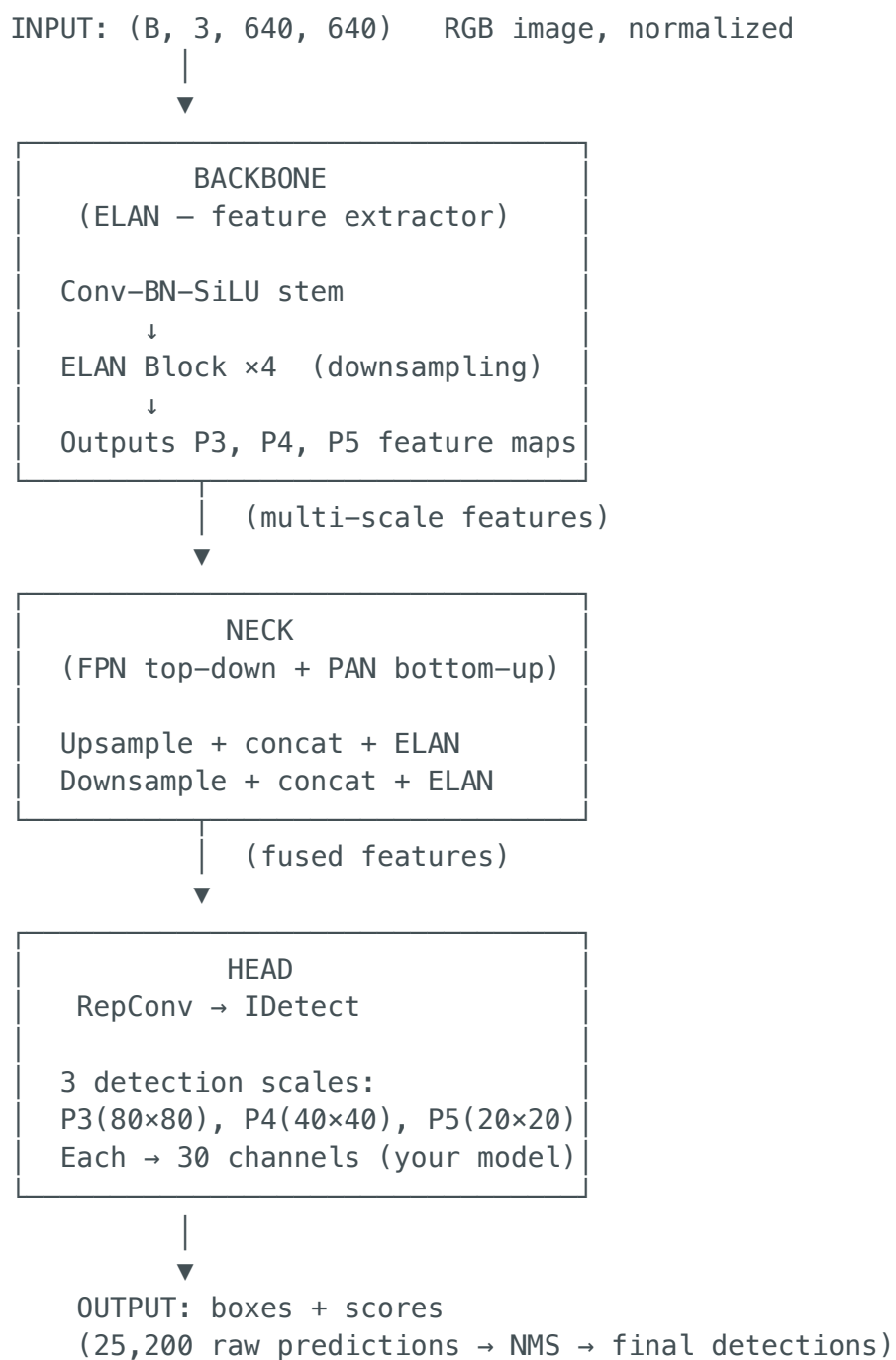
---

## YOLO Evolution (Quick Context)

---

YOLOv1 (2016) → single grid, one scale  
YOLOv2 (2017) → anchor boxes added  
YOLOv3 (2018) → 3-scale detection (FPN)  
YOLOv4 (2020) → CSP backbone + PANet neck  
YOLOv5 (2020) → PyTorch rewrite  
YOLOv7 (2022) → ELAN backbone + E-ELAN + RepConv head ← YOUR MODEL

# Full YOLOv7 Architecture



# 1. Input Processing

Before entering the network, every image goes through **letterbox resizing**:

```
Original image (any size, e.g. 1080×720)
    ↓
Scale so longest side = 640, preserve aspect ratio
    ↓
Pad short side with grey (114, 114, 114) to make 640×640
    ↓
Normalize: pixel / 255.0 → [0, 1]
    ↓
Subtract ImageNet mean [0.485, 0.456, 0.406]
Divide by ImageNet std [0.229, 0.224, 0.225]
```

## Why letterbox and not just resize?

Squishing an image distorts aspect ratios — a barbell becomes square. Letterbox preserves the true shape of objects so the model sees them as they naturally appear.

# 2. Backbone — ELAN

ELAN = **Efficient Layer Aggregation Network**. Instead of passing features from one layer to the next sequentially, ELAN aggregates (concatenates) outputs from multiple layers simultaneously, giving the model both shallow and deep context at every stage.

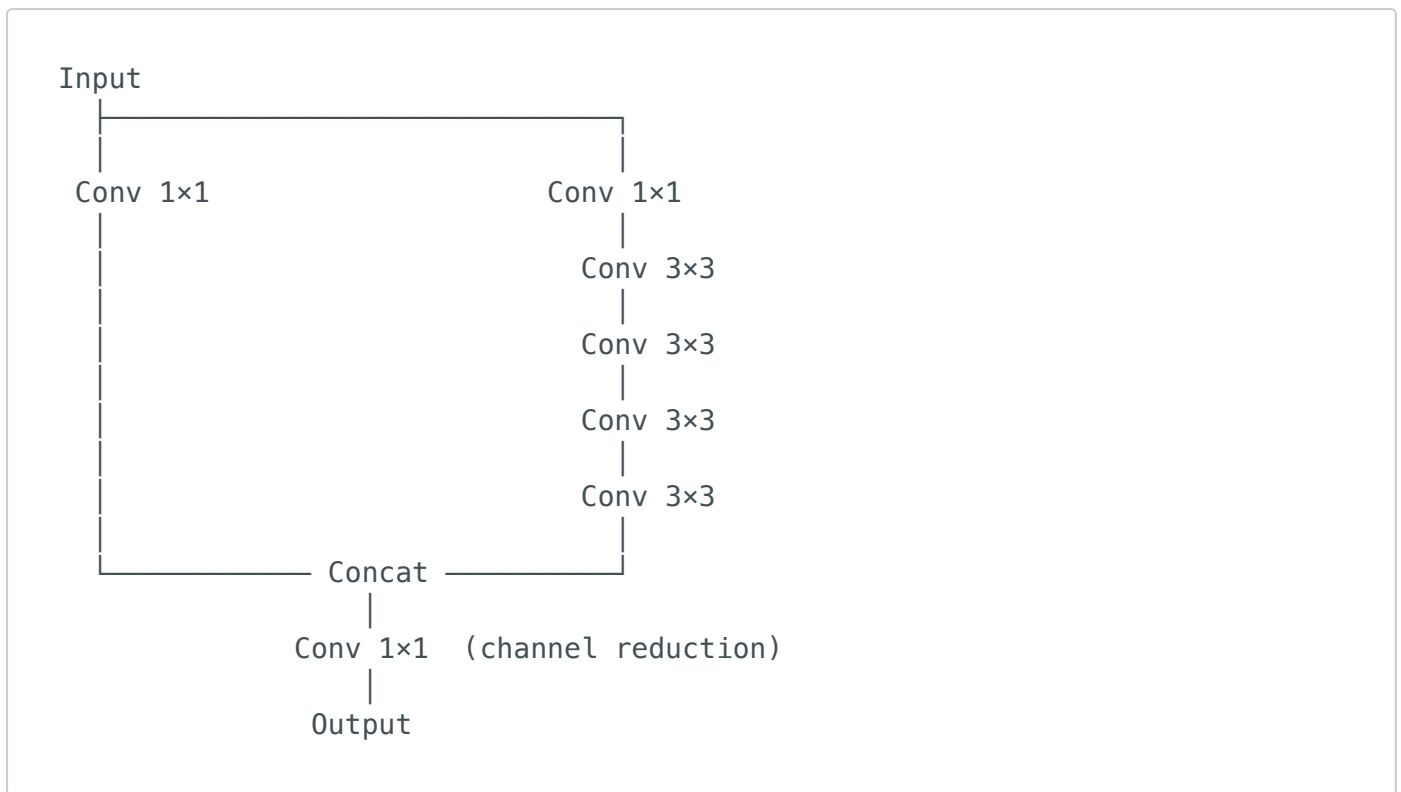
## Basic Building Block: Conv-BN-SiLU

```
Input feature map
    |
Conv2d (learnable filters)
    |
BatchNorm (normalize activations across batch)
    |
SiLU activation:  $f(x) = x \times \text{sigmoid}(x)$ 
    |
Output feature map
```

## Why SiLU instead of ReLU?

| ReLU                               | SiLU                                         |
|------------------------------------|----------------------------------------------|
| $\max(0, x)$ — kills all negatives | $x \times \sigma(x)$ — keeps small negatives |
| Dead neuron problem                | Smooth, non-monotonic                        |
| Hard zero at $x=0$                 | Smooth transition near zero                  |
| Standard choice                    | ~1–2% better accuracy in deep networks       |

## ELAN Block



### Why this matters:

Each path through the block has a different receptive field.

- Short path → local features (edges, textures)
- Long path → global features (shapes, full objects)

Concatenating all paths gives rich multi-scale context at every layer.

## Backbone Stages

| Layer | Operation          | Output Size | Channels |
|-------|--------------------|-------------|----------|
| Input |                    | 640×640     | 3        |
| Stem  | Conv 3×3, stride 2 | 320×320     | 32       |

|         |                   |         |     |      |
|---------|-------------------|---------|-----|------|
| Stage 1 | ELAN + downsample | 160×160 | 64  |      |
| Stage 2 | ELAN + downsample | 80×80   | 128 | ← P3 |
| Stage 3 | ELAN + downsample | 40×40   | 256 | ← P4 |
| Stage 4 | ELAN + downsample | 20×20   | 512 | ← P5 |

**Downsampling** is done by stride-2 convolution (not max-pool) — learnable downsampling that preserves more information.

**P3, P4, P5** are the three feature maps passed to the neck. Each has a different receptive field:

| Feature Map | Spatial Size | Receptive Field | Good For       |
|-------------|--------------|-----------------|----------------|
| P3          | 80×80        | ~52×52 pixels   | Small objects  |
| P4          | 40×40        | ~103×103 pixels | Medium objects |
| P5          | 20×20        | ~206×206 pixels | Large objects  |

## 3. Neck — FPN + PAN

The neck fuses features across all three scales in two passes.

### FPN — Top-Down Path (semantic information flows down)

```

P5 (20×20, 512ch)
|
1×1 Conv (reduce channels)
|
Upsample ×2 → (40×40)
|
Concat with P4 (40×40)
|
ELAN Block → P4_fused (40×40, 256ch)
|
1×1 Conv (reduce channels)
|
Upsample ×2 → (80×80)
|
Concat with P3 (80×80)

```

ELAN Block → P3\_fused (80×80, 128ch)

### Why top-down?

P5 has the largest receptive field — it knows *what* is in the image (semantic knowledge) but has low spatial resolution (poor *where*). FPN passes this semantic knowledge down to the higher-resolution feature maps that know *where* things are precisely.

## PAN — Bottom-Up Path (spatial information flows up)

```
P3_fused (80×80)
|
Stride-2 Conv (downsample) → (40×40)
|
Concat with P4_fused (40×40)
|
ELAN Block → N4 (40×40)
|
Stride-2 Conv (downsample) → (20×20)
|
Concat with P5_fused (20×20)
|
ELAN Block → N5 (20×20)
```

### Why bottom-up too?

FPN alone loses fine spatial information at deeper scales. PAN re-propagates precise localization info back up. Together FPN+PAN means every detection scale has both semantic richness and spatial precision.

## Final Neck Outputs

|    |                |           |                          |
|----|----------------|-----------|--------------------------|
| N3 | (80×80, 128ch) | stride 8  | → detects small objects  |
| N4 | (40×40, 256ch) | stride 16 | → detects medium objects |
| N5 | (20×20, 512ch) | stride 32 | → detects large objects  |

## 4. Head — RepConv + IDetect

# RepConv (Re-parameterized Convolution)

YOLOv7's key innovation in the head — uses structural re-parameterization:

TRAINING TIME:

Input



INFERENCE TIME (after fusion):

Input



The three branches are **mathematically fused** into one 3x3 conv at inference using algebraic re-parameterization:

$$W_{\text{fused}} = W_{3 \times 3} + \text{pad}(W_{1 \times 1}) + \text{pad}(W_{\text{identity}})$$
$$b_{\text{fused}} = b_{3 \times 3} + b_{1 \times 1} + b_{\text{identity}}$$

**Result:** Training = 3 branches (better gradient flow, higher accuracy). Inference = 1 branch (faster, same result). No accuracy trade-off.

## IDetect — The Detection Head

For each of the 3 scales, one IDetect branch:

```
N3 (80×80, 128ch)
|
RepConv
|
Conv2d(128 → 30, kernel=1×1)    30 = 3 anchors × (5 + 5 classes)
|
Output: (B, 30, 80, 80)
```

```
N4 (40×40, 256ch)
|
RepConv
|
Conv2d(256 → 30, kernel=1×1)
|
Output: (B, 30, 40, 40)
```

```
N5 (20×20, 512ch)
|
RepConv
|
Conv2d(512 → 30, kernel=1×1)
|
Output: (B, 30, 20, 20)
```

**Original COCO head:** `Conv2d(in → 255)` because  $3 \times (5+80) = 255$

**Your fine-tuned head:** `Conv2d(in → 30)` because  $3 \times (5+5) = 30$

This is the **only part you replaced** with randomly initialized weights.

---

## 5. Anchor Boxes

---

YOLOv7 uses **9 pre-defined anchor shapes** (3 per scale), generated by k-means clustering on ground-truth box dimensions from your training set.

```
Stride 8 (P3, small objects):    3 smallest anchors
Stride 16 (P4, medium objects):   3 medium anchors
Stride 32 (P5, large objects):    3 largest anchors
```

Each anchor is a (**width, height**) pair in pixels. Typical values for 640×640 input:

```
P3 anchors: (12×16), (19×36), (40×28)
P4 anchors: (36×75), (76×55), (72×146)
P5 anchors: (142×110), (192×243), (459×401)
```

### What anchors do:

Instead of predicting absolute box size (any number from 0–640), the model predicts a *scale factor relative to the anchor*. This is far easier to learn — small corrections rather than arbitrary sizes from scratch.

### Anchor matching during training:

For each ground-truth box, the best-matching anchor is the one with highest IoU when both are centered at the origin. Only matched anchors contribute to the box regression loss.

---

# 6. Prediction Decoding

The raw network output (tx, ty, tw, th, to, tc0...tc4) per anchor is decoded as:

```
For anchor at grid cell (i, j) with anchor size (Pw, Ph) and stride s:

bx = (sigmoid(tx) + i) × s      ← center x in pixels
by = (sigmoid(ty) + j) × s      ← center y in pixels
bw = Pw × exp(tw)                ← width  in pixels
bh = Ph × exp(th)                ← height in pixels

obj_conf  = sigmoid(to)          ← P(object exists here)
class_i   = sigmoid(tc_i)        ← P(class i | object exists)

final_conf = obj_conf × max(class_i)
```

## Why sigmoid for tx, ty?

Constrains the center prediction to [0, 1] — keeps the predicted center inside its own grid cell. Without this, the model could predict a box center that "jumps" to a neighboring cell, which destabilizes training.

## Why exp for tw, th?

Box dimensions must be positive. exp maps any real number → positive. A raw prediction of tw = 0 gives bw = Pw × 1 = Pw (the anchor size itself — a sensible default initialization).

# 7. Total Predictions Per Image

| Scale     | Grid Size | Anchors | Predictions  |
|-----------|-----------|---------|--------------|
| Stride 8  | 80 × 80   | 3       | 19,200       |
| Stride 16 | 40 × 40   | 3       | 4,800        |
| Stride 32 | 20 × 20   | 3       | 1,200        |
| TOTAL     |           |         | 25,200 boxes |

All 25,200 are produced in a **single forward pass**. Most have near-zero objectness and are filtered out by the confidence threshold. Typically 10–50 survive after NMS.

## 8. Loss Function

$$\text{Total Loss} = \lambda_{\text{box}} \times L_{\text{box}} + \lambda_{\text{obj}} \times L_{\text{obj}} + \lambda_{\text{cls}} \times L_{\text{cls}}$$

Your weights:

$$\lambda_{\text{box}} = 0.05$$

$$\lambda_{\text{obj}} = 0.70$$

$$\lambda_{\text{cls}} = 0.30$$

### L<sub>box</sub> — CloU Loss (Complete IoU)

$$\text{CIoU} = 1 - \text{IoU} + (d^2 / c^2) + \alpha \times v$$

Where:

IoU = overlap area / union area

$d^2$  = squared Euclidean distance between predicted and GT centers

$c^2$  = squared diagonal of the smallest enclosing box

$v$  = aspect ratio consistency term

$$= (4/\pi^2) \times (\arctan(w_{\text{gt}}/h_{\text{gt}}) - \arctan(w_{\text{pred}}/h_{\text{pred}}))^2$$

$$\alpha = v / (1 - \text{IoU} + v) \quad (\text{balancing weight})$$

#### Why CloU instead of MSE?

| MSE Loss                              | CloU Loss                           |
|---------------------------------------|-------------------------------------|
| Penalizes x, y, w, h independently    | Treats box as a unified shape       |
| Ignores overlap between boxes         | Directly optimizes IoU              |
| Same loss even if boxes don't overlap | Always sensitive to overlap quality |
| Not scale-invariant                   | Scale-invariant by design           |
| Ignores aspect ratio                  | Penalizes aspect ratio mismatch     |

### L<sub>obj</sub> — Objectness Loss (BCE with IoU target)

```
L_obj = BCE(sigmoid(to), IoU_predicted_vs_GT)
```

The target is not a hard 0 or 1 — it is the **actual IoU** between the predicted box and the ground-truth box. A nearly-correct prediction gets a target close to 1, rewarding approximate detections.

### **Class imbalance:**

For an 80×80 grid × 3 anchors = 19,200 cells. If you have 5 objects in the image, only 5 cells are "positive". 19,195 cells are background. Without correction, the model learns to always predict "no object".

YOLOv7 handles this by weighting positive vs negative objectness contributions separately.

## **L\_cls — Classification Loss (BCE)**

```
L_cls = BCE(sigmoid(tc_i), one_hot_gt_i)
```

Uses BCE (binary cross-entropy per class) rather than softmax cross-entropy — allows multiple classes to be active simultaneously, which handles overlapping objects correctly.

---

## **9. Non-Maximum Suppression (NMS)**

After the forward pass you have 25,200 raw boxes. NMS removes duplicates:

```
Step 1: Filter all boxes with confidence < threshold  
        (conf = 0.001 for mAP evaluation, 0.25 for deployment)
```

```
Step 2: For each class separately:  
        a. Sort remaining boxes by confidence (highest first)  
        b. Take the highest-confidence box → KEEP  
        c. Remove all boxes with IoU > 0.45 with the kept box  
        d. Repeat with the next highest-confidence box
```

```
Step 3: Merge results across all classes → final detections
```

## Why per-class NMS?

A dumbbell and a kettlebell might overlap heavily in the same image. Per-class NMS ensures they don't suppress each other just because their bounding boxes overlap.

## Why `conf=0.001` for mAP?

mAP evaluation requires the full precision-recall curve. Using a very low threshold keeps all possible detections so the curve is computed correctly. Deployment uses a higher threshold (0.25) to show only confident predictions.

---

# 10. What Your Fine-Tuning Changes

---

### PRETRAINED YOLOv7 (80 classes)

Backbone: ELAN (FROZEN A)  
Neck: FPN + PAN (FROZEN A)  
Head Conv2d: (in, 255, 1, 1)  
detect.nc = 80  
detect.no = 85 (80+5)

→

### YOUR FINE-TUNED MODEL (5 classes)

Same weights – preserved  
Same weights – preserved  
Replaced: (in, 30, 1, 1) ← random init  
detect.nc = 5  
detect.no = 10 (5+5)

**Phase A (2 epochs):** Only the 3 new Conv2d layers trained. Backbone frozen.

**Phase B (40 epochs):** All layers trained at 10× lower LR to prevent catastrophic forgetting.

---

# 11. Full Number Summary

---

|                            |             |                       |
|----------------------------|-------------|-----------------------|
| Total parameters:          | 36,501,466  |                       |
| Backbone parameters:       | ~28,000,000 | (ELAN blocks)         |
| Neck parameters:           | ~8,000,000  | (FPN + PAN)           |
| Original head (80 class):  | ~738,048    | (3 × Conv2d, 255 out) |
| Fine-tuned head (5 class): | ~91,248     | (3 × Conv2d, 30 out)  |

|                  |                          |
|------------------|--------------------------|
| Input tensor:    | (1, 3, 640, 640)         |
| Raw predictions: | 25,200 boxes per image   |
| After NMS:       | typically 0–20 boxes     |
| Model size:      | 146 MB on disk (float32) |

# 12. One-Line Summary of Every Component

| Component              | One Line                                                         |
|------------------------|------------------------------------------------------------------|
| <b>Conv-BN-SiLU</b>    | Learn filters → normalize → smooth-activate                      |
| <b>ELAN</b>            | Aggregate features from multiple depths simultaneously           |
| <b>FPN (top-down)</b>  | Pass semantic "what" knowledge from deep to shallow layers       |
| <b>PAN (bottom-up)</b> | Pass spatial "where" knowledge from shallow to deep layers       |
| <b>RepConv</b>         | 3 training branches fused into 1 at inference — speed + accuracy |
| <b>Anchors</b>         | Pre-defined box shapes so model predicts small corrections       |
| <b>sigmoid(tx, ty)</b> | Constrain box center inside its grid cell                        |
| <b>exp(tw, th)</b>     | Ensure predicted box dimensions are always positive              |
| <b>CloU</b>            | Optimize overlap, center distance, and aspect ratio together     |
| <b>BCE objectness</b>  | Score "is there an object?" per cell with IoU as soft target     |
| <b>BCE class</b>       | Score each class independently — handles multi-class overlap     |
| <b>NMS</b>             | Remove duplicate detections per class by greedy IoU suppression  |